

Monitoreo De Temperatura - Humedad - ACC

Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio
Electrónica Digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	4
5. Estructura del código	5
6. Explicación de funciones principales	5
7. Comunicación	5
8. Interfaz de usuario	6
9. Procedimiento de prueba	6
10. Manejo de errores y seguridad	7

1. Identificación del proyecto

- **Nombre del proyecto:** Monitoreo De Temperatura - Humedad - ACC
- **Integrantes del equipo:** Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio

2. Resumen

En esta práctica se diseñó e implementó un sistema de monitoreo IoT utilizando el microcontrolador ESP32 programado en MicroPython. El sistema permite supervisar variables ambientales y de movimiento en tiempo real mediante sensores electrónicos y generar alertas automáticas cuando las variables salen de rangos establecidos.

El proyecto integra un sensor DHT11 para medir temperatura y humedad, un sensor MPU6050 para detectar movimiento y aceleración, un buzzer para alarmas auditivas y un botón de pánico para activación manual de emergencias. Además, el sistema incorpora conectividad WiFi para enviar notificaciones mediante Telegram y alojar un servidor web local para supervisión remota.

El programa implementa múltiples funciones como lectura de sensores, control de alarmas, comunicación inalámbrica y actualización de una interfaz web dinámica. También se emplea programación multihilo para ejecutar simultáneamente el servidor web y el monitoreo principal.

Este proyecto permitió fortalecer conceptos relacionados con sistemas embebidos, Internet de las Cosas (IoT), manejo de periféricos, comunicación inalámbrica, sensores digitales, protocolos de red y programación en tiempo real.

3. Descripción del hardware

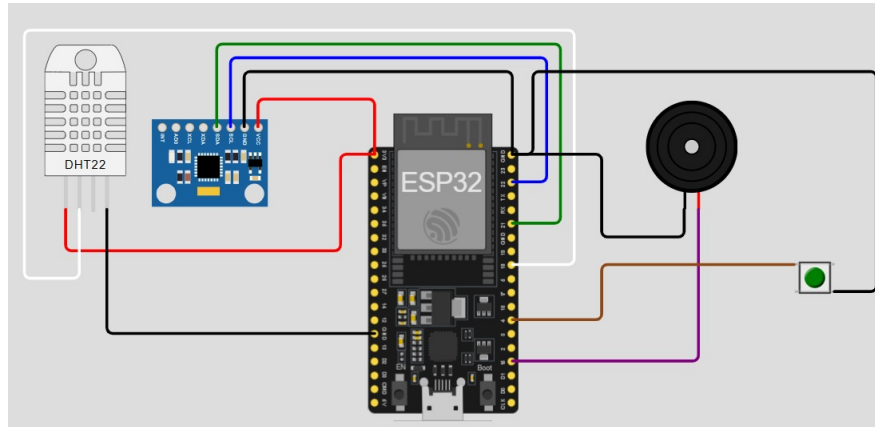
- **Microcontrolador:** ESP32 encargado de ejecutar la lógica del sistema de monitoreo IoT, gestionar sensores, alarmas y comunicación inalámbrica.
- **Entradas:**
 - **Sensor DHT11 (pin 18):** sensor digital utilizado para medir temperatura y humedad ambiental.
 - **Sensor MPU6050 (pines SDA 21 y SCL 22):** módulo acelerómetro y giroscopio utilizado para detectar movimiento y cambios bruscos de aceleración mediante comunicación I2C.
 - **Botón de pánico (pin 4):** pulsador digital utilizado para activar manualmente una alarma de emergencia.
- **Salidas:**
 - **Buzzer (pin 15):** actuador sonoro controlado mediante PWM para indicar diferentes tipos de alarmas.

- **Comunicación inalámbrica:**

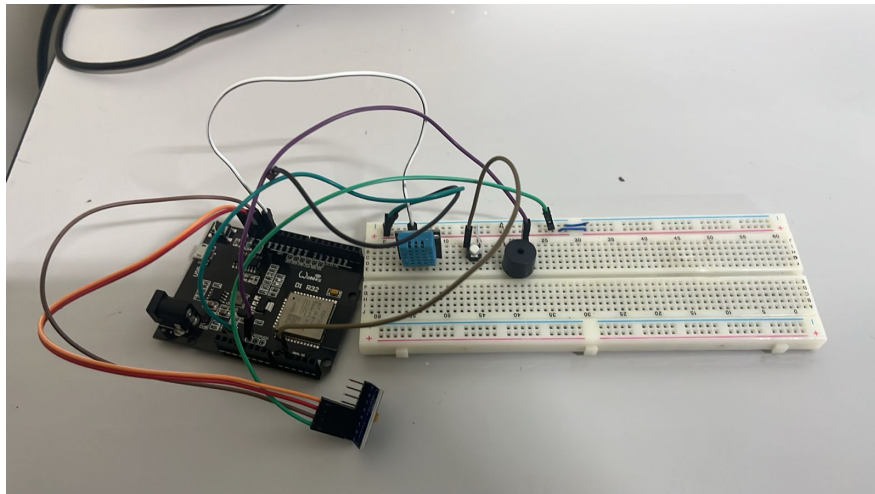
- **WiFi integrado ESP32:** permite la conexión a Internet, acceso al servidor web y comunicación con Telegram.

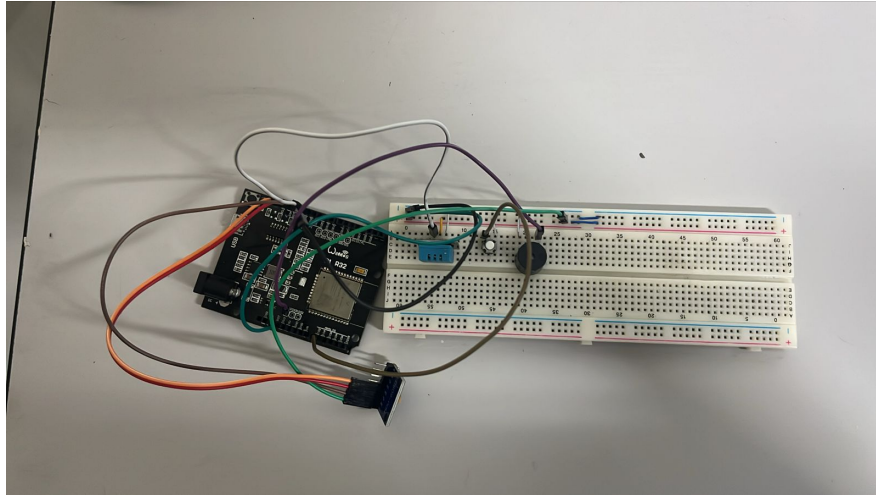
- **Fuente de alimentación:** 3.3V suministrados mediante conexión USB.

Diagrama de conexión:



Montaje físico:





4. Descripción del software

- **Lenguaje:** MicroPython
- **Librerías:** machine, network, socket, time, dht, urequests, imu, _thread
- **IDE:** Thonny

Flujo general:

1. Configuración de pines y periféricos.
2. Inicialización de sensores y buzzer.
3. Configuración de conexión WiFi.
4. Inicialización del bot de Telegram.
5. Inicio del servidor web.
6. Ejecución del ciclo principal.
7. Lectura periódica del sensor DHT11.
8. Lectura del sensor MPU6050.
9. Verificación de límites y alarmas.
10. Activación del buzzer según el tipo de alerta.
11. Envío de notificaciones mediante Telegram.
12. Actualización de interfaz web en tiempo real.

5. Estructura del código

- Archivo principal: `main.py`
- Variables globales:
temperatura actual, humedad actual, estado de movimiento, alarmas, dirección IP y umbrales de funcionamiento.
- Funciones:
 - `send_telegram_alert()` envía mensajes mediante Telegram.
 - `revisar_telegram()` consulta comandos enviados desde Telegram.
 - `wifi_connect()` conecta el ESP32 a la red WiFi.
 - `iniciar_servidor_web()` crea la página web de monitoreo.
 - `gestionar_buzzer()` controla las frecuencias del buzzer según la alarma.

6. Explicación de funciones principales

- `send_telegram_alert()`: Permite enviar mensajes automáticos al usuario utilizando la API de Telegram para reportar alarmas o estados del sistema.
- `revisar_telegram()`: Consulta periódicamente si el usuario envió comandos como `/temp`, `/hum`, `/mov` o `/estado` y responde con la información correspondiente.
- `wifi_connect()`: Realiza la conexión del ESP32 a la red WiFi y obtiene la dirección IP del dispositivo.
- `iniciar_servidor_web()`: Genera un servidor web local que muestra las variables medidas y permite silenciar la alarma desde un navegador.
- `gestionar_buzzer()`: Controla el buzzer utilizando PWM y modifica la frecuencia dependiendo del tipo de alerta detectada.

7. Comunicación

La comunicación en el sistema se realiza mediante:

- I2C: comunicación entre el ESP32 y el sensor MPU6050.
- WiFi: conexión inalámbrica para acceso web y Telegram.
- HTTP: protocolo utilizado para el servidor web.
- API Telegram: envío y recepción de mensajes remotos.

8. Interfaz de usuario

- **Servidor web:** muestra temperatura, humedad, estado del movimiento y alarmas en tiempo real.
- **Telegram:** permite consultar variables mediante comandos y recibir alertas automáticas.
- **Botón de pánico:** activa manualmente una alarma crítica.
- **Buzzer:** proporciona retroalimentación auditiva dependiendo del evento detectado.
- **Elementos visuales:**
 - Indicador de temperatura.
 - Indicador de humedad.
 - Estado de movimiento.
 - Estado de alarmas.
 - Dirección IP del sistema.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador mediante USB.
2. Abrir el IDE Thonny.
3. Cargar el archivo `main.py`.
4. Ejecutar el programa.
5. Verificar la conexión WiFi.
6. Confirmar la lectura del sensor DHT11.
7. Confirmar la lectura del MPU6050.
8. Generar movimiento brusco para validar alarmas.
9. Presionar el botón de pánico.
10. Verificar el sonido del buzzer.
11. Ingresar desde un navegador a la IP del ESP32.
12. Confirmar la actualización de la página web.
13. Consultar variables mediante Telegram.
14. Validar recepción de alertas automáticas.

10. Manejo de errores y seguridad

- Uso de estructuras `try-except` para evitar fallos del sistema.
- Verificación de conexión WiFi antes de iniciar servicios.
- Control de tiempos mediante `time.sleep()` y `ticks_ms()`.
- Validación de límites máximos y mínimos de temperatura y humedad.
- Manejo de excepciones en Telegram y servidor web.
- Activación manual de emergencia mediante botón de pánico.
- Reinicio automático de lectura en caso de error temporal.
- Uso de variables de estado para evitar conflictos de alarmas.